

Универзитет у Београду
Факултет организационих наука
Катедра за софтверско инжењерство



Семинарски рад из предмета
Напредне јава технологије

Веб апликација за резервацију Електричних возила у Spring Boot и React
окружењу

Ментор: проф. др Илија Антовић

Студент: Андреј Драгутиновић

Број индекса: **2022/0016**

Београд, 2026.

Садржај

1.	Вербални опис.....	3
	Основни циљеви:	3
	Корисничке улоге и функционалности	3
	Архитектура и технологије	4
	Закључак	4
2.	Технологије.....	5
2.1.	Spring Boot	5
2.2.	Spring Boot Web	5
2.3.	Spring Data JPA	6
2.4.	React.js.....	6
2.5.	Maven	6
2.6.	MySQL	7
3.	Инсталација и општи опис структуре клијентског дела апликације	8
	Инсталација и покретање пројекта	8
	Коришћене технологије	9
	Структура пројекта	9
	10	
	Опис главних компоненти	11
4.	Случајеви коришћења.....	13
	21	
5.	Структура пројекта	22
6.	Архитектура пројекта	23
7.	Закључак	24

1. Вербални опис

Пројекат „Electric Vehicle Rental“ представља савремену веб апликацију за управљање системом изнајмљивања електричних возила. Развијен је по клијент–сервер архитектури, при чему је позадински део (backend) имплементиран у Java Spring Boot оквиру, а кориснички интерфејс (frontend) реализован помоћу React.js.

Главни циљ система је дигитализација и аутоматизација процеса изнајмљивања

Основни циљеви:

- **За администраторе:** омогућити ефикасно управљање возилима, типовима возила, корисницима и резервацијама кроз једноставан административни интерфејс.
- **Технички циљ:** обезбедити стабилан, сигуран и скалабилан систем, изграђен на принципима модерне софтверске архитектуре (REST API, сервисна слојевитост, реактиван интерфејс).

Корисничке улоге и функционалности

Систем подржава један кориснички тип – администратора, који управља целокупним системом изнајмљивања.

Основне функционалности администратора:

- **Управљање возилима (CRUD):** Филтрирање и претрага по моделу возила и по цени по дану.
- **Управљање корисницима (CRUD):** Регистрација корисника (ЈМБГ, име, презиме, епошта, телефон, град); филтрирање, претрага, ажурирање и брисање.
- **Управљање вожњама:** Креирање нових вожњи са дефинисаним датумима преузимања и враћања возила; брисање и претрага.

Архитектура и технологије

Backend:

- Језик: Java
- **Оквир:** Spring Boot
- База података: MySQL
- **ORM:** Hibernate (преко Spring Data JPA)
- Архитектура: REST API

Frontend:

- **Језик:** JavaScript
- **Оквир:** React.js
- **Структура:** компонентно оријентисана архитектура са сервисима (api.js) за комуникацију са backend-ом

UI: чист и минималистички дизајн, табеларни приказ података и интерактивне форме

Закључак

„Electric Vehicle Rental“ представља комплетан систем за управљање изнајмљивањем електричних возила, који покрива све оперативне процесе — од евиденције возила и корисника до контроле и управљања возњама.

Захваљујући стабилној архитектури и јасно дефинисаним валидационим правилима, апликација је поуздана, једноставна за употребу и спремна за даља проширења, као што су: систем за обраду плаћања, аутентификација више корисничких улога и напредно извештавање о искоришћености возног парка.

2. Технологије

2.1. Spring Boot

Spring Boot је оквир заснован на Јави који омогућава брз и једноставан развој backend апликација. У оквиру пројекта Electric Vehicle Rental, Spring Boot се користи за имплементацију REST API сервиса који повезује React frontend са MySQL базом података.

Омогућава аутоматску конфигурацију, минималан број XML подешавања и модуларни приступ кроз starter зависности. У комбинацији са Spring Data JPA, овај оквир управља ентитетима као што су Vozilo, Korisnik и Voznja.

У пројекту се користи за:

- креирање REST контролера за CRUD операције (возила, корисници, возње);
- повезивање слојева контролера, сервиса и репозиторијума;
- аутоматску конфигурацију сервера (уграђени Tomcat).

2.2. Spring Boot Web

Spring Boot Web модул омогућава креирање веб апликација и REST API сервиса. Обрађује HTTP захтеве и шаље JSON одговоре ка клијенту.

У систему Electric Vehicle Rental, овај модул се користи за:

VoziloController, KorisnikController, VoznjaController;

- руковање HTTP методама (GET, POST, PUT, DELETE) за управљање подацима;
- комуникацију између React клијента и Spring Boot сервера;
- локално покретање апликације путем уграђеног Tomcat сервера без ручног деплоја.

2.3. Spring Data JPA

Spring Data JPA представља део Spring екосистема који поједностављује рад са базом података кроз ORM (Object-Relational Mapping). Омогућава рад са ентитетима без класичних SQL упита.

У пројекту, овај модул:

Vozilo, TipVozila, Korisnik, Voznja, Grad;

VoziloRepository, VoznjaRepository) аутоматски генерише имплементације метода попут `findAll()`, `save()`, `deleteById()`;

`search()` за напредну претрагу;

- обезбеђује интеграцију са MySQL базом и аутоматско креирање табела.

2.4. React.js

React.js је библиотека за изградњу динамичних корисничких интерфејса заснована на компонентној архитектури. У овом пројекту је коришћен са фокусом на једноставност, модуларност и брзу реакцију апликације.

React се у пројекту користи за:

VehiclesPage, RidePage, RideHistoryPage, KorisnikPage;

- формирање интерактивних форми за унос, измену и брисање података;
- динамичко освежавање табела без поновног учитавања странице

2.5. Maven

Мавен је алат за управљање зависностима и изградњу Јава пројеката. Користи се у Електриц Вехицле Рентал апликацији за управљање свим библиотекама, плугинима и процесом компилације.

Главна конфигурација се налази у `pom.xml` фајлу, где су дефинисане зависности као што су:

- `spring-boot-starter-web`
- `spring-boot-starter-data-jpa`
- `mysql-connector-j`
- `lombok`

Мавен обезбеђује:

- једноставно укључивање свих потребних библиотека;
- изградњу извршне ЈАР датотеке;
- аутоматско управљање зависностима и њиховим верзијама.

2.6. MySQL

MySQL је релациона база података која складишти све податке у систему — возила, типове возила, клијенте, градове и резервације.

У пројекту Electric Vehicle Rental, MySQL обезбеђује:

- перзистенцију података кроз ентитете;
- повезаност табела преко страних кључева (нпр. клијент–град, возило–тип возила, резервација–возило);
- спровођење правила целокупности података, укључујући ON DELETE CASCADE за аутоматско брисање зависних података.

Уз комбинацију ових технологија, Electric Vehicle Rental обезбеђује поуздан, брз и ефикасан систем за управљање изнајмљивањем електричних возила, са потпуном интеграцијом између бацкенд-а и фронтенд-а.

3. Инсталација и општи опис структуре клијентског дела апликације

Инсталација и покретање пројекта

Пројекат „Electric Vehicle Rental“ развијен је коришћењем React библиотеке и Vite алаткама.

1. Креирање пројекта:

```
npm create vite@latest electric-vehicle-rental -- --template react
```

(Ова команда креира нови React пројекат коришћењем Vite build алата.)

2. Улазак у фолдер пројекта:

```
cd electric-vehicle-rental
```

3. Инсталација основних зависности:

```
npm install
```

(Ovom komandom se instaliraju sve potrebne biblioteke koje su navedene u package.json fajlu.)

4. Инсталација додатних библиотека:

```
npm install axios react-router-dom
```

5. Покретање пројекта:

```
npm run dev
```

(Апликација се подиже на локалном серверу.)

Након ових корака, апликација је покренута на локалној адреси (на пример <http://localhost:5173/>). Vite је омогућио брзо развојно окружење са аутоматским освежавањем при свакој измени у коду.

Коршћене технологије

- **React** — за развој компонентног интерфејса.
- **JavaScript** — језик за имплементацију логике апликације.
- **Axios** — за HTTP комуникацију са бацкенд-ом.
- **React Router** — за управљање навигацијом унутар апликације.

Структура пројекта

```

└─ back [Electric-Vehicle-Rental-Web-App-Full-Stack-Application master]
  └─ src/main/java
    > com.mycompany.mavenprojectpokusaj2
    > com.mycompany.mavenprojectpokusaj2.controller
    > com.mycompany.mavenprojectpokusaj2.dto
    > com.mycompany.mavenprojectpokusaj2.dto.impl
    > com.mycompany.mavenprojectpokusaj2.entity
    > com.mycompany.mavenprojectpokusaj2.entity.impl
    > com.mycompany.mavenprojectpokusaj2.mapper
    > com.mycompany.mavenprojectpokusaj2.mapper.impl
    > com.mycompany.mavenprojectpokusaj2.repository
    > com.mycompany.mavenprojectpokusaj2.repository.impl
    > com.mycompany.mavenprojectpokusaj2.security
    > com.mycompany.mavenprojectpokusaj2.servise
  > src/main/resources
  > src/test/java
  > src/test/resources
  > JRE System Library [jdk-21.0.11.10-hotspot]
  > Maven Dependencies
  > target/generated-sources/annotations
  > target/generated-test-sources/test-annotations
  > Referenced Libraries
  > .settings
  > src
  > target
  > .classpath
```

- ▼ src
 - > assets
 - ▼ components
 - ⊞ AuthPage.jsx
 - ⊞ Dashboard.jsx
 - ⊞ GradoviPage.jsx
 - ⊞ KorisnikPage.jsx
 - ⊞ Navbar.jsx
 - ⊞ RideHistoryPage.jsx
 - ⊞ RidePage.jsx
 - ⊞ VehiclesPage.jsx
 - ⊞ VehicleTypesPage.jsx
 - ▼ contexts
 - ⊞ AuthContext.jsx
 - ⊞ VehicleTypesContext.jsx
 - ▼ services
 - | JS api.js
 - # App.css
 - ⊞ App.jsx
 - # index.css
 - ⊞ main.jsx
- ⊞ eslint.config.js
- <> index.html
- { } package-lock.json
- { } package.json
- ① README.md

Опис главних компоненти

App.jsx - Главна компонента која дефинише све руте у апликацији користећи React Router. Садржи логику за заштиту рута — проверава да ли је корисник пријављен пре него што му дозволи приступ страницама као што су `VehiclesPage`, `RidePage` и `KorisnikPage`.

main.jsx - Улазна тачка апликације која рендерује App компоненту у DOM преко `ReactDOM.createRoot`. Овде се иницијализује глобални контекст аутентификације.

AuthPage.jsx - Компонента која обједињује пријаву и регистрацију корисника. Корисник уноси корисничко име и лозинку, а по успешној пријави JWT токен се чува у `AuthContext` и корисник се преусмерава на `Dashboard`.

Dashboard.jsx - Почетна страница апликације након пријаве. Приказује преглед система и нуди брз приступ свим секцијама апликације.

VehiclesPage.jsx - Страница која приказује сва електрична возила у систему. Омогућена је претрага и филтрирање по типу возила. Користи `useEffect` за иницијално учитавање, а филтрирање се врши локално без новог API позива

RidePage.jsx - Страница за креирање нове вожње. Корисник бира возило и град, а тајмер мери трајање вожње у реалном времену. Цена се рачуна аутоматски на основу `senaPoMinutu` и трајања.

RideHistoryPage.jsx - Страница која приказује историју свих вожњи у табеларном приказу. Паралелно учитава историју, возила и градове коришћењем `Promise.all`, чиме се минимизује број захтева ка серверу.

KorisnikPage.jsx — Страница за управљање корисницима система. Претрага се активира реактивно — свака промена у пољу за унос аутоматски покреће нови API позив захваљујући `search` параметру у зависностима `useEffect` хука.

GradoviPage.jsx — Страница за управљање листом градова. Градови се користе као референтни подаци при регистрацији корисника. Омогућено је додавање нових и брисање постојећих градова.

VehicleTypesPage.jsx — Страница за управљање типовима електричних возила (нпр. тротинет, бицикл, електрични аутомобил). Сваки тип возила има назив и опис.

Navbar.jsx — Горња навигациона трака присутна на свим страницама након пријаве. Приказује корисничко име и дугме за одјаву. `Logout` брише токен из контекста и преусмерава на `/auth`.

AuthContext.jsx — React контекст за глобално управљање аутентификацијом. Чува JWT токен и податке о пријављеном кориснику и чини их доступним свим компонентама.

.

VehicleTypesContext.jsx — React контекст који централизује податке о типовима возила. Учитава типове јединпут при покретању апликације и чини их доступним компонентама, без поновних API позива. Садржи fallback вредности у случају грешке.

api.js — Централни сервисни фајл који обједињује сву HTTP комуникацију са backend-ом коришћењем Axios библиотеке. Interceptor аутоматски додаје Authorization хедер са JWT токеном. Ако сервер врати статус 401, корисник се аутоматски одјављује.

4. Случајеви коришћења

За следеће случајеве коришћења ћемо дати детаљан опис:

CK1	Registracija korisnika
CK2	Prijava korisnika
CK3	Kreiraj voznju
CK4	Pretrazi voznju
CK5	Pretrazi vozilo
CK6	Prikazi vozilo
CK7	Prikazi tip vozila
CK8	Prikazi gradove
CK9	Pretrazi korisnika

5. Приказ функционалности апликације кроз код клијентског дела апликације

У овом поглављу описани су кључни JSX фајлови frontend-а — странице и компоненте које чине кориснички интерфејс Electric Vehicle Rental апликације, са примерима стварног кода.

App.jsx

Главна компонента апликације која дефинише све руте коришћењем React Router-а. Садржи компоненту ProtectedRoute која проверава да ли постоји важећи JWT токен пре дозволе приступа заштићеним страницама. Ако токен не постоји, корисник се аутоматски преусмерава на /auth.

```
function App() {
  return (
    <AuthProvider>
      <VehicleTypesProvider>
        <Router>
          <Routes>
            <Route path="/auth" element={<AuthPage />} />
            <Route path="/dashboard" element={<ProtectedRoute><Dashboard /></ProtectedRoute>} />
            <Route path="/vehicles" element={<ProtectedRoute><VehiclesPage /></ProtectedRoute>} />
            <Route path="/ride" element={<ProtectedRoute><RidePage /></ProtectedRoute>} />
            <Route path="/history" element={<ProtectedRoute><RideHistoryPage /></ProtectedRoute>} />
            <Route path="/vehicle-types" element={<ProtectedRoute><VehicleTypesPage /></ProtectedRoute>} />
            <Route path="/gradovi" element={<ProtectedRoute><GradoviPage /></ProtectedRoute>} />
            <Route path="/korisnici" element={<ProtectedRoute><KorisnikPage /></ProtectedRoute>} />
            <Route path="/" element={<Navigate to="/dashboard" />} />
          </Routes>
        </Router>
      </VehicleTypesProvider>
    </AuthProvider>
  );
}
```

AuthPage.jsx

Компонента која обједињује пријаву и регистрацију корисника. Функција handleSubmit шаље POST захтев ка backend-у, чува токен у AuthContext и преусмерава корисника на Dashboard. Грешке се приказују испод форме.

```
const handleSubmit = async (e) => {
  e.preventDefault();
  setError('');
  try {
    const endpoint = isLogin ? 'auth/login' : 'auth/register';
    const payload = isLogin
      ? { username: formData.username, password: formData.password }
      : { username: formData.username, email: formData.email, password: formData.password };
    const response = await api.post(endpoint, payload);
    login(response.data.token, response.data.user);
    navigate('/dashboard');
  } catch (err) {
    setError(err.response?.data?.message || 'An error occurred');
  }
};
```

AuthContext.jsx

React контекст за глобално управљање аутентификацијом. Функција login чува токен и корисничке податке у localStorage, а logout их брише и преусмерава корисника на страницу за пријаву.

```
const login = (token, user) => {
  localStorage.setItem('token', token);
  localStorage.setItem('user', JSON.stringify(user));
  setToken(token);
  setUser(user);
};

const logout = () => {
  localStorage.removeItem('token');
  localStorage.removeItem('user');
  setToken(null);
  setUser(null);
};
```

api.js

Централни сервисни фајл који обједињује сву HTTP комуникацију са backend-ом користећи Axios. Interceptor аутоматски додаје Authorization header са JWT токеном за сваки захтев. Ако сервер врати статус 401, корисник се аутоматски одјављује.

```
const API_BASE_URL = '/api/';

const api = axios.create({
  baseURL: API_BASE_URL,
});

api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      localStorage.removeItem('token');
      localStorage.removeItem('user');
      window.location.href = '/auth';
    }
    return Promise.reject(error);
  }
);
```


VehiclesPage.jsx

Страница која приказује сва електрична возила у систему. Користи `useEffect` за иницијално учитавање података, а филтрирање по типу возила се врши локално без новог API позива.

```
useEffect(() => {
  const fetchData = async () => {
    try {
      const vehicleResponse = await api.get('vozilo');
      setAllVehicles(vehicleResponse.data);
      setFilteredVehicles(vehicleResponse.data);
      const types = [...new Set(vehicleResponse.data.map(v => v.tipVozilaID))];
      setVehicleTypes(types);
    } catch (err) {
      console.error('Error fetching vehicles:', err);
    }
  };
  fetchData();
}, []);

const handleFilter = () => {
  if (selectedType === '') {
    setFilteredVehicles(allVehicles);
  } else {
    setFilteredVehicles(allVehicles.filter(v => v.tipVozilaID.toString() === selectedType));
  }
};
```

RidePage.jsx

Страница за креирање нове возње. Корисник бира возило и град, а тајмер мери трајање возње у реалном времену. Цена се рачуна аутоматски на основу `сенаРоMinutu` и трајања.

```
useEffect(() => {
  let interval;
  if (isRiding) {
    interval = setInterval(() => {
      setCurrentDuration(Math.floor((Date.now() - startTime) / 1000));
    }, 1000);
  }
  return () => clearInterval(interval);
}, [isRiding, startTime]);

const handleFilterVehicles = () => {
  if (selectedType === '') {
    setFilteredVehicles(allVehicles);
  } else {
    setFilteredVehicles(allVehicles.filter(v => v.tipVozilaID.toString() === selectedType));
  }
  setSelectedVehicle('');
};

const handleClearFilter = () => {
  setSelectedType('');
  setFilteredVehicles(allVehicles);
  setSelectedVehicle('');
};

const handleStart = () => {
  if (!selectedVehicle || !selectedCity) return;
  const confirmStart = window.confirm('Are you sure you want to start the ride?');
  if (confirmStart) {
    setIsRiding(true);
    setStartTime(Date.now());
  }
};
```

RideHistoryPage.jsx

Страница која приказује историју свих вожњи. Паралелно учитава историју, возила и градове користећи `Promise.all`, чиме се минимизује број round-trip позива ка серверу.

```
useEffect(() => {
  const fetchData = async () => {
    try {
      const [historyResponse, vehiclesResponse, citiesResponse] = await Promise.all([
        api.get('voznja/history'),
        api.get('vozilo'),
        api.get('grad')
      ]);
      setRideHistory(historyResponse.data);
      setVehicles(vehiclesResponse.data);
      setCities(citiesResponse.data);
    } catch (err) {
      console.error('Error fetching data:', err);
      setError('Failed to load ride history');
    } finally {
      setLoading(false);
    }
  };

  if (user) {
    fetchData();
  }
}, [user]);
```

KorisnikPage.jsx

Страница за управљање корисницима система. Претрага се активира реактивно — свака промена у пољу за унос аутоматски покреће нови API позив захваљујући `search` параметру у зависностима `useEffect` hook-a.

```

useEffect(() => {
  const fetchKorisnici = async () => {
    setLoading(true);
    try {
      const response = await api.get(`korisnici/search?username=${encodeURIComponent(search)}`);
      setKorisnici(response.data);
      setError('');
    } catch (err) {
      setError('Greška prilikom učitavanja korisnika.');
```

Navbar.jsx

Горња навигациона трака присутна на свим страницама након пријаве. Приказује корисничко име и дугме за одјаву. Logout брише токен из контекста и преусмерава на /auth.

```

const Navbar = () => {
  const { user, logout } = useAuth();
  const navigate = useNavigate();

  const handleLogout = () => {
    logout();
    navigate('/auth');
  };

  const userLabel = user && typeof user === 'object' ? user.username || user.email || user.id : user;

  return (
    <nav className="navbar">
      <div className="navbar-brand">
        <Link to="/dashboard">Rent a Bike</Link>
      </div>
      <div className="navbar-menu">
        <Link to="/dashboard">Dashboard</Link>
        <Link to="/vehicles">Vehicles</Link>
        <Link to="/ride">Ride</Link>
        <Link to="/history">History</Link>
        <Link to="/vehicle-types">Tipovi Vozila</Link>
        <Link to="/gradovi">Gradovi</Link>
        <Link to="/korisnici">Korisnici</Link>
      </div>
      <div className="navbar-user">
        <span>{userLabel}</span>
        <button onClick={handleLogout}>Logout</button>
      </div>
    </nav>
  );
};
```

VehicleTypesContext.jsx

React контекст који централизује податке о типовима возила. Учитава типове једном при покретању апликације и чини их доступним свим компонентама без поновних API позива. Садржи и fallback вредности у случају грешке.

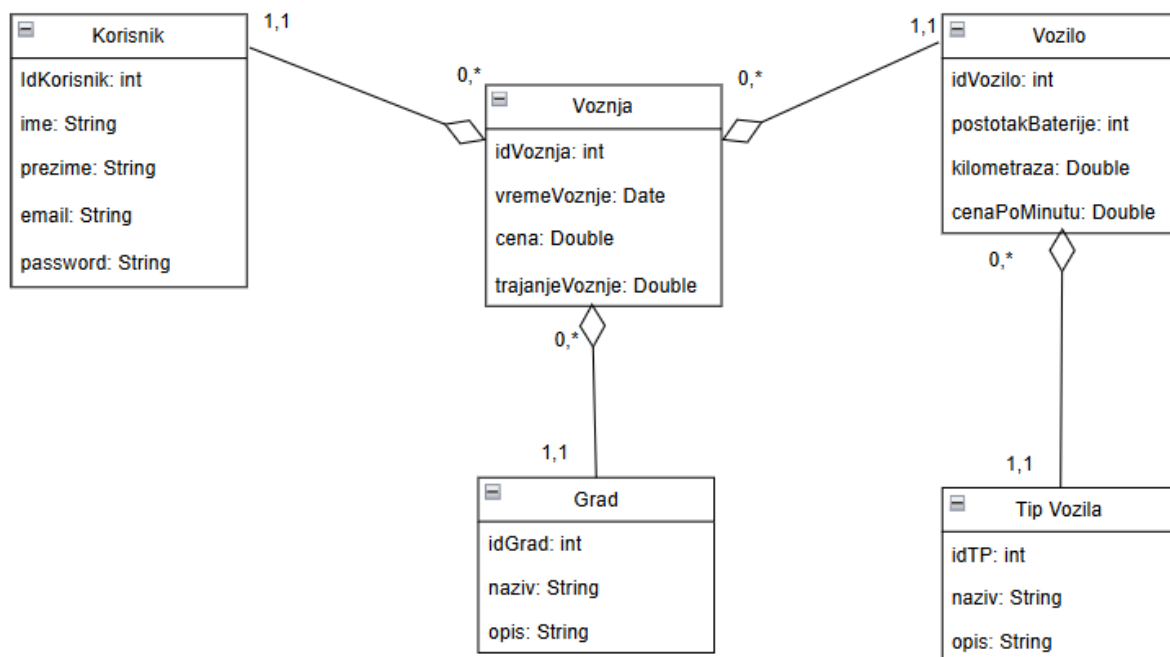
```
export const VehicleTypesProvider = ({ children }) => {
  const [vehicleTypeNames, setVehicleTypeNames] = useState(fallbackTypeNames);

  useEffect(() => {
    const fetchTypes = async () => {
      try {
        const response = await api.get('tipvozila');
        const typeMap = { ...fallbackTypeNames };
        if (response.data && response.data.length > 0) {
          response.data.forEach(type => {
            typeMap[type.id] = type.naziv;
          });
        }
        setVehicleTypeNames(typeMap);
      } catch (err) {
        console.error('Error fetching vehicle types:', err);
        setVehicleTypeNames(fallbackTypeNames);
      }
    };
    fetchTypes();
  }, []);

  return (
    <VehicleTypesContext.Provider value={{ vehicleTypeNames }}>
      {children}
    </VehicleTypesContext.Provider>
  );
};
```

6. Структура пројекта

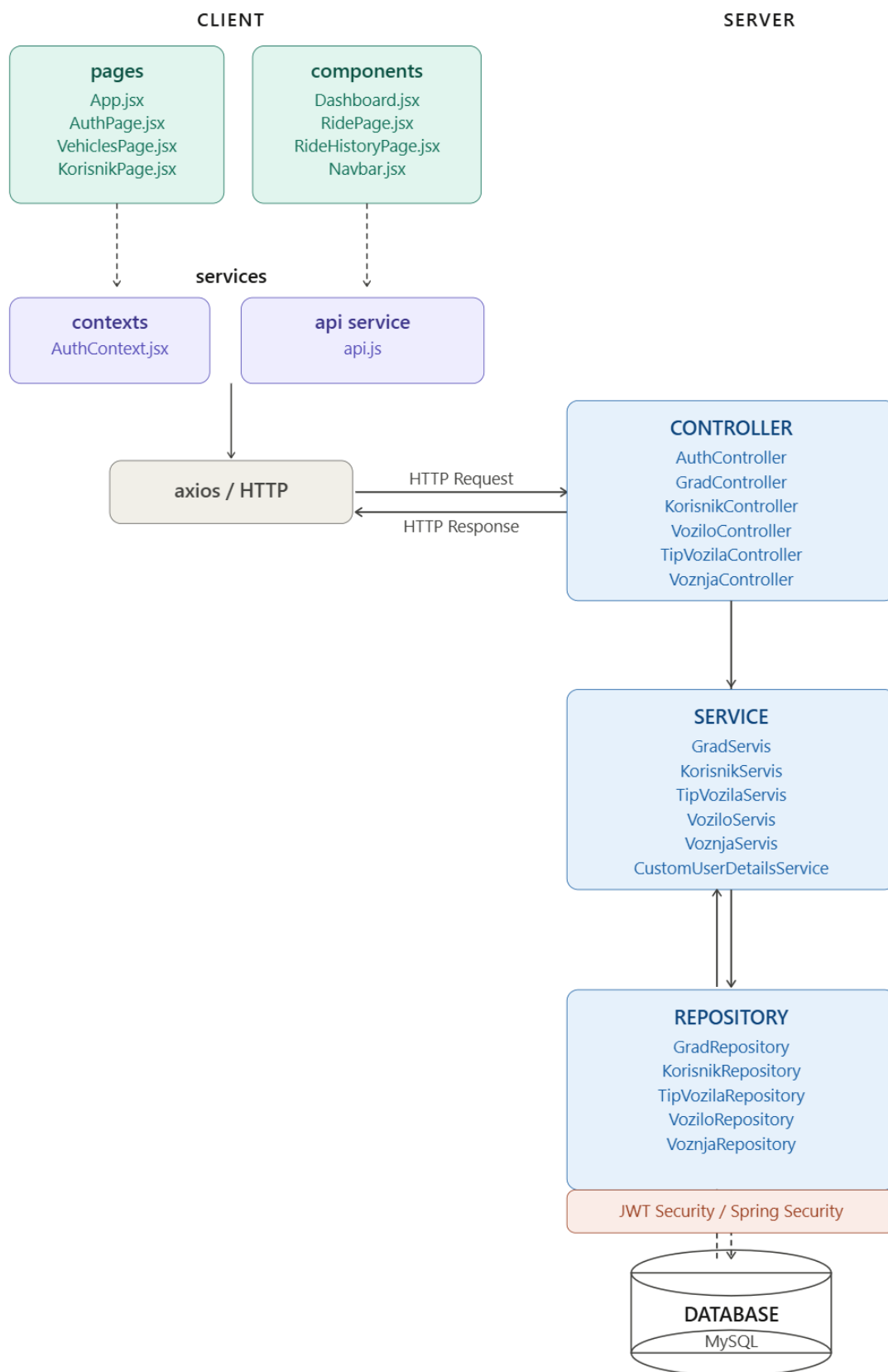
Концептуални модел:



На основу концептуалног модела направљен је следећи релациони модел:

1. Korisnik (idKorisnik, ime, prezime, email, password)
2. Vozilo (idVozilo, postotakBaterije, kilometraza, cenaPoMinutu, idTipVozila)
3. TipVozila (idTP, naziv, opis)
4. Grad (idGrad, naziv, opis)
5. Voznja (idVoznja, vremeVoznje, cena, trajanjeVoznje, idKorisnik, idVozilo, idGrad)

7. Архитектура пројекта



8. Закључак

Овај пројекат показује да комбинација Spring Boot-а и React-а обезбеђује поуздан backend и брз, интерактиван кориснички интерфејс. У поређењу са класичним Java desktop или конзолним апликацијама, React омогућава динамичке форме, тренутне реакције система и пријатније корисничко искуство.

Spring Boot обезбеђује REST API, валидацију података и јасно дефинисане грешке, које се у React делу апликације обрађују помоћу try/catch блока и прикажу као поруке или упозорења.

За складиште података коришћен је MySQL, са JPA/Hibernate мапирањем ентитета (Korisnik, Vozilo, TipVozila, Grad, Vožnja). Ограничења у бази (UNIQUE, FK, CASCADE) осигуравају интегритет података. Maven је коришћен за управљање зависностима и процесом изградње апликације.

Пројекат представља добру основу за даљи развој. Могућа унапређења укључују:

- увођење WebSocket/STOMP функционалности за реална освежавања података;
- управљање улогама и дозволама корисника (нпр. администратор, оператер);
- евиденцију активности (audit log) за праћење промена у систему;
- вишејезичну подршку (i18n) за интернационализацију апликације;
- интеграциона и end-to-end тестирања за већу поузданост система;
- Docker/CI/CD аутоматизацију за лакше подешавање и испоруку апликације;
- унапређење календара доступности возила ради боље прегледности и планирања изнајмљивања.

Захваљујући стабилној архитектури заснованој на Spring Boot и React технологијама, као и јасно дефинисаним слојевима (контролер, сервис, репозиторијум), апликација „Electric Vehicle Rental“ је скалабилна, одржива и спремна за даља проширења у складу са реалним пословним потребама.